Software Requirements Specification (SRS)

Project: Detach

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to define the functional and non-functional requirements for "Detach," a digital well-being and focus-enhancement mobile application. This document serves as the foundational blueprint for development and outlines the system's intended behavior.

1.2 Document Conventions

- **MVP (Minimum Viable Product):** The initial, baseline version of the application containing only the absolute core features necessary for deployment.
- **Strict Mode:** A specialized application state that requires user authentication (via a local PIN) to alter or terminate an active focus session.
- **Target Platform:** Refers to the operating system the app is being built for (Android for the MVP, with iOS architecture considered for future iterations).

1.3 Intended Audience

This document is intended for the lead developer, project supervisors, and future contributors within the open-source community to understand the project's scope, architecture, and core philosophy.

1.4 Project Scope

Detach is a minimalist utility application designed to help users minimize digital distractions by temporarily restricting access to user-selected applications via a focus timer.

In-Scope: The MVP will be developed exclusively for Android using the Flutter framework. All user data (timer history, application preferences) will be stored locally on the device utilizing an SQLite database. The project will be open-source, 100% free to use, and devoid of advertisements or paywalls.

Out-of-Scope: The application explicitly excludes cloud data synchronization, user account creation/authentication, and any form of gamification (e.g., streaks, points, leaderboards) to maintain a strictly utilitarian, distraction-free environment.

2. Overall Description

2.1 Product Perspective

Detach is a standalone mobile application. It does not rely on external cloud servers, web APIs, or cross-device synchronization. All operations, including system-level app blocking and data storage, occur locally on the user's Android device.

2.2 User Classes and Characteristics The primary users are individuals seeking to reduce screen time and improve concentration.

- **Students & Academics:** Require focused blocks of time for studying, utilizing color-coded tags to track different subjects.
- **Professionals:** Need distraction-free periods for deep work, using the focus log as a digital diary of productivity.
- **General Users:** Individuals looking to break the habit of subconscious scrolling through intentional friction. Users are expected to have basic smartphone literacy but desire an interface that is completely free of visual clutter, complex onboarding, or gamified mechanics.

2.3 Operating Environment

- **Operating System:** Android (Minimum SDK version to be determined, likely targeting modern Android versions that support UsageStatsManager or Accessibility Services for app blocking).
- **Database:** Local SQLite database for relational data management (storing focus session history, custom tags, and notes).

2.4 Core Product Functions

The application's environment revolves around three core pillars:

1. **The Enforcer (App Blocking):** Intercepting and preventing the launch of user-specified distracting applications.
2. **The Timer (Focus Sessions):** A customizable countdown mechanism that dictates the duration of the block, with an optional "Strict Mode" protected by a local PIN.
3. **The Focus Log (Digital Diary):** A minimalist calendar view that allows users to review past focus sessions. Users can assign color-coded tags (e.g., "Coding," "Studying") and short notes to sessions prior to starting the timer, creating a visual history of their productivity without gamified streaks.

2.5 Design and Implementation Constraints

- The application must be developed using the Flutter framework.
- The application must function entirely offline.
- The UI must adhere to a strict minimalist design language, avoiding unnecessary animations, pop-ups, or gamified elements.

3. Functional Requirements

3.1 Module: Focus Session Management

- **FR-1.1:** The system shall allow the user to set a custom countdown timer for a focus session (e.g., inputting hours and minutes).
- **FR-1.2:** The system shall allow the user to input a short text note and select a color-coded activity tag (e.g., "Studying," "Coding") prior to starting the timer.
- **FR-1.3:** The system shall display the remaining time actively counting down on the main screen during a session.
- **FR-1.4:** The system shall provide a standard "Cancel Session" button to terminate an active focus session early.
- **FR-1.5:** The system shall hide or disable the "Cancel Session" button if the user initiates the session in "Strict Mode."

3.2 Module: App Blocking and Restrictions

- **FR-2.1:** The system shall retrieve and display a list of all installed applications on the user's Android device.
- **FR-2.2:** The system shall allow the user to select and save specific applications to a "Blocked List."
- **FR-2.3:** The system shall actively monitor app usage while a focus timer is running.
- **FR-2.4:** The system shall immediately intercept and prevent the user from accessing any application on the "Blocked List" while a focus session is active.
- **FR-2.5:** The system shall display a minimalist "Block Screen" (e.g., a simple message reminding them to focus) when a blocked app is intercepted.
- **FR-2.6:** The system shall maintain a hardcoded "Emergency Whitelist" of essential system applications (e.g., the native Phone Dialer) that can never be added to the Blocked List, regardless of user settings.
- **FR-2.7:** The system shall feature an "Always Allowed" settings menu where the user can manually exempt specific apps (e.g., SMS, Calculator, Maps) from ever being blocked, even during Strict Mode.

3.3 Module: Focus Log & Digital Diary (Calendar View)

- **FR-3.1:** The system shall record the date, duration, user note, and color tag of every successfully completed focus session in the local SQLite database.
- **FR-3.2:** The system shall provide a Calendar View displaying the current month.
- **FR-3.3:** The system shall display a small visual indicator (e.g., a colored dot corresponding to the tag) on calendar days that contain completed focus sessions.
- **FR-3.4:** The system shall display a detailed summary (duration, note, and activity type) of a specific day's sessions when the user taps on a date in the Calendar View.

3.4 Module: Security and Preferences

- **FR-4.1:** The system shall allow the user to set up a local, numeric PIN code in the settings menu.
- **FR-4.2:** The system shall require the user to successfully input their PIN to prematurely cancel a "Strict Mode" focus session.

- **FR-4.3:** The system shall allow the user to permanently delete their entire focus log history from the local database.

4. Non-Functional Requirements

4.1 Privacy and Data Security

- **NFR-1.1 (Zero Transmission):** The application shall operate with a strict offline-first, local-only architecture. No user data, including installed app lists, blocked app preferences, or focus log history, shall ever be transmitted to external servers or third-party analytics services.
- **NFR-1.2 (Local Encryption):** The user's "Strict Mode" PIN shall be securely hashed and stored locally on the device (e.g., using Flutter Secure Storage) rather than as plain text.

4.2 Performance and Resource Management

- **NFR-2.1 (Instant Interception):** The native background service responsible for app monitoring must detect and intercept a blocked application within 500 milliseconds of the user attempting to open it, ensuring a seamless "block" screen before the distracting app's UI loads.
- **NFR-2.2 (Battery Efficiency):** Because the app requires continuous background monitoring during an active focus session, the background service must be highly optimized to consume minimal battery power (e.g., utilizing efficient Android APIs like UsageStatsManager rather than constant screen polling).

4.3 Usability and User Experience (UX)

- **NFR-3.1 (Minimalist UI):** The user interface must adhere to strict minimalist design principles, utilizing ample whitespace, clear typography, and an absence of unnecessary animations, pop-ups, or gamified visual clutter.
- **NFR-3.2 (Dark Mode):** The application must fully support a native "Dark Mode" theme to reduce eye strain, which is a critical feature for users focusing in low-light environments.
- **NFR-3.3 (Accessibility):** All core screens must support dynamic text sizing and standard screen readers (e.g., Android TalkBack) to ensure accessibility for all users.

4.4 Reliability and Robustness

- **NFR-4.1 (Background Persistence):** The native Android background service must be robust enough to automatically restart or maintain its state if the Android operating system attempts to aggressively kill it to save memory.
- **NFR-4.2 (Graceful Degradation):** If the user revokes the necessary system permissions (e.g., Accessibility or App Usage permissions) while a timer is running, the app must pause the timer and immediately display a full-screen prompt explaining how to re-enable the required permissions to continue.

5. External Interface Requirements

5.1 User Interfaces (UI) The application UI shall be built using Flutter's Material Design widgets, customized to fit a strict minimalist aesthetic. The core navigation shall consist of the following primary views:

- **The Focus Screen (Home):** A centralized, large countdown timer with a prominent "Start" button. Prior to starting, it features input fields for a session note and a dropdown/selector for color-coded tags.
- **The Shield Screen:** A dedicated menu to view installed applications and toggle them onto the "Blocked List" or "Always Allowed" whitelist.
- **The Log Screen:** A standard monthly calendar view where days with completed sessions are visually marked. Tapping a day opens a modal or bottom sheet displaying the session history (duration, tag, note).
- **The Settings Screen:** A secondary menu for establishing the Strict Mode PIN, managing global preferences, and accessing the "Buy Me a Coffee" donation link.

5.2 Software Interfaces Because the application requires deep system integration, it must interface with the following external software components:

- **Android Native APIs:** The Flutter application will use Platform Channels (Dart-to-Kotlin/Java communication) to interface with Android system services. Specifically, it will utilize android.app.usage.UsageStatsManager or an AccessibilityService to monitor foreground app activity and trigger the blocking mechanism.
- **Local Database:** The application will interface with an embedded SQLite database (sqflite package in Flutter) to execute CRUD (Create, Read, Update, Delete) operations for the focus log and user preferences.

5.3 Hardware Interfaces The application requires standard touchscreen input native to Android devices. No special hardware interfaces (e.g., camera, microphone, Bluetooth) are required.

5.4 Communications Interfaces None. The application operates strictly offline. It shall not interface with any external network protocols, cloud databases, or web-based APIs.